

C R E D E N C E

CREDIT RISK INTELLIGENCE PLATFORM

Function Reference & Glossary

What every function in the codebase does, and what every
buzzword and finance term in the project actually means.

PART 1 Every function, all ~150 of them — pipeline (Python) + dashboard (TypeScript)

PART 2 Glossary — credit-risk & finance terms, statistics/ML terms, engineering terms

Two references in one document

Part 1 goes file by file through the entire codebase, in real pipeline order, and explains what every function actually does — what it takes in, what it hands back, and why it exists. Part 2 is a glossary: every finance term, statistics/ML term, and engineering buzzword used anywhere in the project or its documentation, explained in plain language. Use Part 1 to answer 'what does this function do,' and Part 2 to answer 'what does that word mean' the moment it comes up.

PART 1

Function Reference

Every function in the pipeline and the dashboard, file by file, in the order data actually flows through the system.

The pipeline (Python)

`src/config.py`

Loads config.yaml and turns it into the values every other module reads.

`load()` -> dict

Reads config.yaml off disk and resolves every relative file path in it (the raw CSV, the DuckDB file, the model file) into an absolute path, so the rest of the code never has to worry about what directory it's being run from.

`sql_params(cfg)` -> dict[str, str]

Flattens the nested config dictionary into a flat string map — the exact set of `${...}` placeholder values the SQL templating engine (`src/db.py`) substitutes into the .sql files.

`src/db.py`

A thin wrapper around DuckDB: open a connection, run a .sql file with placeholder substitution.

`connect(db_path, read_only=False)`

Opens (or creates) the DuckDB database file at the given path and returns a live connection.

`_render(sql_text, params)` -> str

Does the actual `${key}` -> value substitution in a block of SQL text, using a small regex and the params map — raises a clear error if a placeholder has no matching value.

`run_sql_file(con, name, params)` -> list

Reads a .sql file, strips comments so a semicolon inside a comment can't break statement-splitting, substitutes placeholders, splits on ';', and executes each statement in order — returns the last statement's result so the caller can print a summary.

`show(rows, headers=None)`

Pretty-prints up to 25 rows of a query result to the console, with a '+N more' note if there are more — just a console-output convenience used after each pipeline stage.

`sql/01_raw.sql`, `sql/02_staging.sql`, `sql/03_features_outcome.sql`, `sql/06_marts.sql`, `schema/star_schema.sql`

Pure SQL — no functions in the programming-language sense, but each file is itself one unit of work: load raw data, clean it, split it into leakage-safe tables, compute Expected Loss and the serving marts, and (`star_schema.sql`) build the BI-facing fact/dimension views.

`src/features.py`

The feature contract — what the model is and isn't allowed to see.

`engineer(df)` -> pd.DataFrame

Adds two derived columns (log annual income, loan-to-income ratio) and replaces any infinite values with NaN so the downstream scaler never chokes on a bad division.

`build_preprocessor()` -> `ColumnTransformer`

Builds the unfit scikit-learn preprocessing pipeline: median-impute-then-scale for numeric columns, most-frequent-impute-then-one-hot-encode for categorical columns.

`assert_no_leakage(df)`

Checks a dataframe against the FORBIDDEN column list and raises an error if any of them are present — the runtime safety net on top of the SQL-level table separation.

src/train.py

Fits the PD model, calibrates it, benchmarks it, and writes every metric to disk.

`_quiet_blas()`

A context manager that silences harmless floating-point warnings some OpenBLAS builds emit during model fitting, so the console output stays readable.

`_load_frame(con)` -> `pd.DataFrame`

Pulls the full modelling frame from the database, runs feature engineering on it, and asserts no leakage before returning it.

`_estimator(kind, cfg)` -> `Pipeline`

Builds an unfit preprocessing-plus-classifier pipeline for either 'hgb' (HistGradientBoosting) or 'logistic', using the hyperparameters in config.yaml.

`_fit_calibrated(kind, X, y, cfg)`

Fits the chosen estimator and, unless calibration is turned off, wraps it in `CalibratedClassifierCV` so its output probabilities are genuinely calibrated, not just a ranking score.

`_auc_of(kind, Xtr, ytr, Xte, yte, cfg)` -> `(float, model)`

Fits a calibrated model of the given kind and returns its test-set AUC plus the fitted model itself — used to get the secondary model's score for the three-way comparison.

`_grade_benchmark(bench_train, bench_test, y_train, y_test)` -> `dict`

Builds the simplest possible baseline: for each Lending Club grade, use its historical average default rate (learned on train) as the predicted PD, then scores that baseline on test — this is what the real model has to beat.

`_lightgbm_delta(X_train, y_train, X_test, y_test, cfg)` -> `dict`

Optionally fits a LightGBM model as a second comparison point, if the lightgbm library is available in the current environment; returns a note explaining why not, if it isn't.

`main()` -> `float`

Runs the whole training stage end to end: load data, fit the primary model, fit the secondary model and the grade benchmark, run the DeLong test between them, compute every metric, write metrics.json and model.joblib, print a summary, and return the test AUC.

src/evaluate.py

The shared statistical toolkit — metrics, significance testing, and plots.

`ks_statistic(y_true, y_score)` -> `float`

Computes the Kolmogorov-Smirnov statistic: the largest gap between the cumulative distribution of good loans and bad loans as you sort by score.

`decile_table(y_true, y_score, k=10) -> pd.DataFrame`

Sorts the population into 10 equal-sized groups by predicted score and reports, per group, the count, mean predicted PD, observed default rate, and lift over the overall base rate.

`_midrank(x) -> np.ndarray`

A helper for the DeLong test — computes the 'mid-rank' of every value in an array, correctly handling ties, which the DeLong formula requires.

`delong_roc_test(y_true, score_a, score_b) -> dict`

The real DeLong test: given two models' scores on the same sample, computes both AUCs, their difference, a z-statistic, and a p-value for whether that difference is real or could be chance.

`summary(y_true, y_score) -> dict`

Bundles AUC, Gini, KS, Brier score, base rate, and sample size into one dictionary — the standard metric block written for both the train and test split.

`plot_calibration(y_true, y_score, path, title)`

Draws and saves a predicted-vs-observed calibration chart (a scatter plus a 45-degree reference line) as a PNG.

`plot_decile_lift(dec, path, title)`

Draws and saves a bar-plus-line chart of the decile table — observed default rate as bars, predicted PD as a line — as a PNG.

`_save(fig, path)`

A small shared helper: makes sure the output folder exists, saves a matplotlib figure at a fixed resolution, and closes it to free memory.

`write_metrics(metrics, path)`

Serializes a metrics dictionary to a JSON file, creating the parent folder if needed.

src/score.py

Batch-scores every loan in the mature window.

`main() -> int`

Loads the trained model, runs `predict_proba` over every loan in `mart_loan_features` (not just the test split, so the whole book gets scored), and writes `loan_id` plus predicted PD to a new `mart_scores` table; returns the number of loans scored.

src/lgd.py

Loss-given-default — estimated from real recoveries, segmented by grade, credibility-weighted.

`estimate(con, oot_cutoff) -> dict`

Computes the single, flat, portfolio-wide LGD from charged-off TRAIN-period loans: 1 minus the recovery rate (principal repaid plus recoveries, over funded amount).

`estimate_by_grade(con, oot_cutoff, lgd_flat) -> list[dict]`

Repeats the same recovery-rate calculation per Lending Club grade, then blends each grade's own raw estimate with the flat portfolio figure using Buhlmann credibility weighting, so grades with little charge-off history don't get a noisy, untrustworthy number.

`write_grade_table(con, by_grade)`

Writes the credibility-weighted per-grade LGD values into a small DuckDB table (`lgd_by_grade`) that the marts stage joins against to compute Expected Loss.

`resolve(cfg, con=None) -> float`

Returns the single effective flat LGD for this pipeline run — either the config's fixed assumption, or the cached data-estimated value from `lgd.json`.

`main() -> float`

Runs the full LGD stage: computes the flat estimate, the per-grade credibility-weighted estimates, writes both to disk/DB, prints a summary, and returns the flat value.

sql/06_marts.sql

Expected Loss and the serving marts (documented under Part 1's SQL entry above) — computed once, reused everywhere downstream.

src/backtest.py

Tests an approval policy against real, historical cash outcomes — not the model's own predictions about itself.

`_load_test(con) -> pd.DataFrame`

Pulls every terminal, test-split loan along with its real cash-flow fields (payments received, recoveries) needed to compute what actually happened.

`_curves(df, fico_min, rf, s, T, b, n_points=60) -> pd.DataFrame`

The core calculation: for 60 possible PD cut-offs, computes both the model's projected profit and the loan book's ACTUAL realised profit if that cut-off had been used, plus predicted vs actual default rate at each point.

`_plot(curve, path)`

Draws and saves the two-panel backtest chart: profit vs approval rate, and predicted vs actual default rate, both against the real curve.

`main() -> dict`

Runs the backtest stage end to end: builds the curve, finds the model's optimum and the truly-best (realised) optimum, computes the regret from following the model's choice instead, writes `backtest.json`, and prints a summary.

src/fairness.py

Disparate-impact / 4/5ths-rule check, with a real statistical confidence interval.

`_load(con) -> pd.DataFrame`

Pulls each test-split loan's predicted PD plus the three proxy demographic fields available in this data (income band, region, home ownership) and whether it was approved under the current policy.

`_bootstrap_air_ci(df, col, approved, keep_groups, alpha, n_boot=2000, seed=42)`

Runs a stratified bootstrap: resample the approved/declined outcome within each group thousands of times, recompute the adverse-impact ratio each time, and take the percentile interval — answers how confident we can be the ratio is really below 0.80, not just below it by chance.

`_by(df, col, approved, min_group=500) -> dict`

For one dimension (e.g. income band), computes the approval rate per group, the adverse-impact ratio, its bootstrap confidence interval, whether it fails the 4/5ths rule, and the approved-book default rate per group.

`main() -> dict`

Runs the fairness stage across all three dimensions, writes `fairness.json`, and prints a pass/flag summary for each.

src/monitor.py

A real, retrospective population-stability (PSI) check between the train and test cohorts.

`_psi(reference, comparison, n_bins=10) -> dict`

Computes the Population Stability Index between two distributions: bins the reference population into deciles, compares what share of each population falls in each bin, and sums the weighted log-ratio — the standard PSI formula.

`_band(psi) -> str | None`

Converts a raw PSI number into a plain-language read: 'stable', 'moderate shift', or 'material shift', using standard risk-monitoring thresholds.

`main() -> dict`

Runs PSI on the model's own score and its five strongest features, comparing the train and test cohorts, writes monitor.json, and prints a summary.

src/insight.py

Two supplementary real analyses: global feature importance, and a report of the segmented LGD.

`_feature_importance(con, cfg, sample_n=15000, n_repeats=5) -> dict`

Runs scikit-learn's permutation_importance on the actual trained model over a 15,000-loan sample of the test set: shuffles one feature at a time and measures how much the model's AUC drops, five times per feature for stability.

`_lgd_by_grade(con, cfg) -> dict`

Reads the canonical credibility-weighted LGD-by-grade values (via src/lgd.py) and packages them for reporting — does not recompute them independently, so there's one source of truth.

`main() -> dict`

Runs both analyses, writes insight.json, and prints the top features and the LGD-by-grade table.

src/explain.py

Real per-loan attribution on the actual trained model, for a small sample of real test loans.

`_typical_values(pop, feat_cols) -> dict`

Computes the 'typical' value of every model feature across the test population — the median for numeric fields, the most common value for categorical ones.

`_explain_loan(model, feat_cols, row_df, typical, top_n=6) -> dict`

For one real loan, computes its actual predicted PD, then re-predicts PD after resetting each feature to its typical value one at a time, and returns the top contributing features ranked by how much each one moved the PD.

`main(n_per_band=3, seed=20240611) -> dict`

Samples 3 real test loans per risk band, runs _explain_loan on each, writes explanations.json, and prints a short summary.

run_pipeline.py

The orchestrator — runs every stage, in order, from one command.

`_banner(name)`

Prints a '=== stage name ===' header to the console between stages.

stage_raw / stage_stage / stage_features / stage_lgd / stage_train / stage_score / stage_marts
/ stage_export / stage_backtest / stage_fairness / stage_monitor / stage_insight /
stage_explain / stage_check

One small dispatch function per pipeline stage — each one calls into the real module (`src/train.py`, `src/lgd.py`, etc.) that does the actual work; this file's job is only sequencing, not logic.

`run_preflight(cfg, params)`

A one-off diagnostic: for every issue-month in the raw data, reports what share of loans from that month have reached a final outcome — used once, up front, to decide where the maturity window should be drawn.

`main(argv)`

The command-line entry point: parses which stages to run (or 'all'), opens one shared database connection, and runs each requested stage's dispatch function in order.

`tests/test_evaluate.py`, `tests/test_sim_core.py`, `tests/test_methodology.py`

Real assertions, not smoke tests — each `test_*` function asserts a specific, meaningful property (the DeLong test correctly detects a real difference and correctly detects no difference; the simulator's profit identity holds component by component and has a genuine interior optimum; LGD credibility weighting shrinks thin segments and trusts thick ones; PSI reads near-zero for identical distributions and high for a real shift; the fairness bootstrap CI flags a real gap and doesn't flag a synthetic non-gap). Each file's own `__main__` block runs every `test_*` function it finds and reports pass/fail.

The dashboard (CREDENCE, TypeScript/React)

`src/main.tsx`, `src/App.tsx`

Entry point and routing.

`App()`

The route table — renders the persistent AppShell (nav rail) around whichever page's route matches the current URL.

`src/lib/format.ts`

Small, consistent number-formatting helpers used on every page.

`usdCompact(n, opts)`

Formats a dollar figure compactly — \$8.2B, \$524.2M, \$12.7K — choosing the right suffix automatically.

`usd(n)`

Formats a dollar figure in full, with thousands separators.

`count(n)`

Formats a plain integer with thousands separators (loan counts, etc.).

`pct(n, decimals=1)`

Formats a 0-1 fraction as a percentage string.

`pp(n, decimals=1)`

Formats a signed change in percentage POINTS (not percent) — e.g. '+1.6 pp'.

`signedPct(n, decimals=1)`

Formats a 0-1 fraction as a signed percentage, with a leading + for positive values.

`fixed(n, decimals=2)`

A plain fixed-decimal formatter, used for things like AUC (0.691) that aren't currency or percentages.

`ordinalDecile(i)`

Turns a 0-based decile index into '1st', '2nd', ... '10th' for chart labels.

`src/lib/risk.ts`

Risk-band definitions and their colours — the one place both are defined.

`bandFor(pd, cuts=DEFAULT_CUTS)`

Given a PD and a set of cut-offs, returns which risk band (Low/Medium/High/Critical) that PD falls into.

`src/lib/curve.ts`

Safe interpolation along the real backtest curve.

`interpCurve(points, pdCut)`

Linearly interpolates the backtest curve at any PD cut-off between two measured points — and clamps to the measured range at the edges, so the simulator can never show a value the pipeline didn't actually backtest.

`curveExtent(points, key)`

Returns the [min, max] of one field across the whole backtest curve — used for chart axis scaling.

src/lib/scales.ts

Minimal chart-scaling math for the hand-built SVG charts (no charting library).

`linear(domain, range)`

Builds a linear scale function mapping a data domain (e.g. 0-100%) onto a pixel range (e.g. 40-800px), with a matching `invert()` for turning a pixel position back into a data value.

`ticks(min, max, target=5)`

Chooses a small set of 'nice' round tick values covering a range, the way a real chart axis would.

`niceMax(v)`

Rounds a value up to the nearest 'nice' number (1, 2, 5, 10x a power of ten) for a clean axis ceiling.

`smoothPath(pts)`

Converts a list of points into a smooth SVG path string using Catmull-Rom-to-Bezier interpolation, for the curved line charts.

`linePath(pts)`

The straight-line (non-smoothed) equivalent of `smoothPath`, for charts that shouldn't curve.

src/app/settings.tsx

Session-level display settings, shared via React context.

`SettingsProvider({ children })`

Holds density, reduced-motion, and risk-band cut-off state for the whole app, and provides it to every page via context.

`useSettings()`

The hook every page calls to read (and update) the current settings — density, reduced-motion, risk-band cut-offs.

src/components/primitives.tsx

The base design-system components every page is built from.

`Delta({ value, format, invert, neutral })`

A small, colour-coded up/down indicator for a signed change — colours green or red depending on whether an increase is good or bad news for that particular metric (invert flips that logic for risk metrics, where 'up' is bad).

`Stat({ label, figure, sub, spark })`

The standard KPI tile: a label, a big figure, an optional sub-line, and an optional inline sparkline.

`Section({ title, desc, aside, children })`

The standard page-section wrapper: a heading, a description, an optional right-aligned control area (aside), and the section's content.

`Card({ title, note, aside, variant, children })`

The standard content card used throughout the dashboard, with optional title/note/aside.

`Seg({ options, value, onChange, labels })`

A segmented control (a row of toggle buttons where exactly one is active) — used for metric/range toggles throughout.

`DemoBadge({ what })`

The small 'DEMO DATA' pill shown on every synthetic panel, with a tooltip explaining what's synthetic.

`Switch({ on, onToggle, label })`

A simple accessible on/off toggle control.

`RiskPill({ band })`

A small coloured pill showing a risk band name in that band's colour.

`Crumb({ children })`

A small breadcrumb-style inline element with a chevron icon.

src/components/blocks.tsx

Composite, page-level visual blocks built from the primitives.

`RiskSpectrum({ bands })`

The Overview hero visualization: funded exposure by risk band as one proportional bar, plus a legend and a plain-language readout sentence.

`FairnessSignal({ dims })`

Computes which fairness dimensions fail the 4/5ths rule and renders a real, colour-coded flag row for each one that does — renders nothing if none fail.

`sevHex(s)`

Maps an alert's severity (High/Medium/Low) to its display colour.

`AlertList({ items })`

Renders a vertical list of monitoring alerts with a severity dot, title, detail, and metadata row.

`AlertTimeline({ items })`

The dated, vertical-line timeline variant of the alert list, used on the Monitoring page's 'Emerging risk' section.

src/components/charts.tsx

Every chart in the app, hand-built as SVG — no charting library dependency.

`niceMax(v)`

A local, finer-grained version of the scales.ts helper of the same name, tuned for this file's chart axes.

`Sparkline({ values, width, height, color })`

A tiny inline line chart with no axes, used inside Stat tiles.

`LineChart({ x, series, height, yFormat, yFrom0, markers, smooth })`

The main trend-chart component: draws one or more series with gridlines, axis labels, a hover crosshair, and a tooltip.

`BarStrip({ rows, valueFormat, color, onSelect, highlight })`

A vertical list of horizontal comparison bars (e.g. exposure by loan purpose), with optional click-to-select and a highlight prop for cross-filtering.

`BarPair({ groups, a, b, yFormat, height })`

A grouped vertical bar chart comparing two series side by side per category (e.g. approved vs declined by income band), with a hover tooltip.

`CalibrationChart({ points, height })`

The predicted-vs-observed scatter plot with a 45-degree reference line and per-point hover detail, used on the Monitoring page.

`ContributionBars({ rows, expanded, onToggle })`

Diverging plus/minus bars from a zero baseline, used for the borrower-level 'why is this risky' panel; clicking a row expands its explanatory note.

`Distribution({ bins, color, height, valueFormat, highlight })`

A tinted histogram (e.g. loan count by FICO band), with an optional highlight for the currently-filtered bin.

`src/components/AppShell.tsx`

The persistent navigation rail and shared page header.

`Item({ to, label, icon })`

One clickable row in the navigation rail.

`AppShell({ children })`

The outer layout: the nav rail on the left, the current page's content on the right.

`PageHead({ title, lede, aside })`

The standard page title block used at the top of every page.

`src/components/Icon.tsx`, `src/components/DataTable.tsx`, `src/components/ThresholdSimulator.tsx`, `src/components/BorrowerDrawer.tsx`

Supporting components.

`Icon({ name, size, strokeWidth })`

Renders one hand-drawn SVG icon by name from a small shared icon set — no icon-library dependency.

`DataTable({ ... })`

A generic, reusable sortable and searchable table component — the Borrowers page's applicant list is built on top of this.

`ThresholdSimulator()`

The interactive centrepiece of the Decisions page: a slider bound to `interpCurve()`, with a live stat grid and a real profit-vs-approval-rate chart that updates as the slider moves.

`Def({ label, value })`

A small label/value pair row used inside the borrower detail drawer.

`BorrowerDrawer({ borrower, onClose })`

The slide-over panel showing one borrower's full financial/credit/loan profile, its contribution-bar explanation, and a decision narrative.

`src/pages/*.tsx`

The seven pages — each one composes the primitives, blocks, and charts above around real data (`src/data/real.ts`) and demo data (`src/data/demo.ts`).

`Overview()`

The landing page: KPI row, the risk-spectrum hero, the real fair-lending signal (shown only when a dimension actually fails), a risk-over-time trend, a 'where risk concentrates' breakdown with click-through into Portfolio, and the demo alert feed.

`ConcentrationStrips({ rows, onSelect })`

A small helper component used only within Overview.tsx — renders one dimension's concentration breakdown (e.g. by grade) as a set of comparison rows.

Portfolio()

The exposure/risk-by-segment page: six real, cross-filtering dimensions (grade, purpose, FICO band, income band, vintage, region) plus the segmented LGD-by-grade panel.

Borrowers()

The applicant-level page: a sortable/searchable table over the 220 synthetic borrowers, opening a detail drawer via a URL parameter.

Decisions()

The policy-decision page: the real backtest-driven KPI row, the ThresholdSimulator, and approval/default/loss comparisons by income band and by decision.

Monitoring()

The model-health page: trend tiles, the real train-vs-test model comparison, calibration, the real global feature-importance ranking, the real PSI panel, the real per-loan attribution sample, and the benchmark comparison.

psiTone(band)

Maps a PSI stability band ('stable'/'moderate shift'/'material shift') to a display colour, used only within Monitoring.tsx.

TrendTile({ ... })

One trend card in the Monitoring page's 'Trends' grid — a label, current value, delta, sparkline, and an optional demo badge.

DataModel()

The accountability page: dataset/model provenance, the full REAL/DEMO panel ledger, and the 'Known limitations' section.

Settings()

Display density, reduced-motion, and risk-band display cut-off controls.

Glossary — Buzzwords & Finance Terms

Every term you'll run into anywhere in this project or its documentation, grouped by category, explained in plain language.

Credit risk & finance terms

PD — Probability of Default

The model's estimated chance that a specific loan ends in default, expressed as a number between 0 and 100%. This is the single most important number in the whole system — every risk band, every dollar figure, and every dashboard panel is ultimately a function of PD. It is a forward-looking ESTIMATE made at or near origination, before anyone knows the real outcome, which is different from the observed default rate (the ACTUAL share of a group that defaulted, measured after the fact, once outcomes are known). A well-built PD model should track the observed rate closely across the population it scores — this project's calibration check exists specifically to verify that.

EAD — Exposure at Default

The dollar amount a lender genuinely stands to lose if a specific loan defaults right now — how much money is actually on the table. In this project EAD is treated as a static number, the amount originally funded at the start of the loan, rather than the loan's current outstanding balance. That's a real simplification worth understanding: a loan that's already had two years of payments made against it has a much smaller true balance at risk than its original funded amount, so using the origination-time figure as EAD systematically overstates exposure for older, more-seasoned loans. A more sophisticated system would model EAD as it changes over the life of the loan, especially for revolving products like credit cards where the outstanding balance can go up as well as down.

LGD — Loss Given Default

Of the money that was actually at risk on a loan that defaults, the share the lender never gets back, expressed as a percentage. If a lender recovers 55 cents on every dollar owed on a defaulted loan (through partial payments already made, or collections afterward), LGD is 45%. It's the piece of the loss calculation that answers 'how bad is a default, once it happens' — as distinct from PD, which answers 'how likely is a default at all.' This project estimates LGD directly from real historical recovery data rather than assuming a textbook number, and segments it by loan grade because riskier grades don't just default more often, they also recover less when they do (44% LGD on the safest grade versus a meaningfully higher figure on the riskiest, once you account for how much data actually backs each estimate).

EL — Expected Loss

The single dollar figure that answers 'how much money do we expect to lose on this loan (or this whole portfolio) on average, before we know the actual outcome.' It's computed as $EL = PD \times EAD \times LGD$: the chance of a bad outcome, times how much money is exposed, times how much of that exposure would actually be lost. It's called 'expected' in the statistical sense — the average outcome across many similar loans, not a prediction about any single one. A lender uses portfolio-wide Expected Loss the way a business uses any other cost forecast: to price products, to set aside capital reserves against future losses, and to judge whether a segment of the book is worth continuing to lend into.

Underwriting

The process of deciding whether to approve a loan application, and on what terms (how much, at what interest rate), based on an assessment of how risky that applicant is. Historically this was a human loan officer's judgment call, often inconsistent from one reviewer to the next; a PD model is what lets that judgment be replaced, or at least backed up, by a consistent, measurable, auditable number applied the same way to every applicant. 'Underwriting

policy' or 'credit policy' refers to the actual rule the lender applies — in this project, a PD cut-off (approve if predicted PD is below some threshold) combined with a minimum credit-score floor.

Origination

The moment a loan is actually created and funded — money changes hands and the loan officially begins. 'Origination-time' is an important qualifier used throughout this project: it means information that genuinely existed and was knowable at the moment the lending decision was made, as opposed to information that only exists because of what happened AFTER the loan was funded (payment history, whether it eventually defaulted, how much was recovered). A model is only honest if it's trained exclusively on origination-time information — using anything from after the fact would be like grading an exam using the answer key, and is exactly what this project's leakage guards are designed to prevent.

Default / charge-off

A loan that has stopped being repaid and that the lender has formally written off as a loss, rather than continuing to carry as a performing asset. 'Charged off' is the specific accounting term for that write-off event. This is a DEFINED, discrete outcome — in this project's data, a loan is only counted as a default once its status is recorded as 'Charged Off' or 'Default,' full stop, not based on any softer judgment call. This matters because the entire PD model is trained to predict exactly this specific, precisely-defined event, not a vaguer notion of 'risky.'

Delinquency

Being behind on scheduled loan payments without yet having reached the more severe, usually terminal, state of default. A loan can be 30, 60, or 90+ days delinquent and still recover — the borrower catches up and the loan goes on to perform normally. This makes delinquency a leading indicator, something that tends to show up before a default does, which is exactly why early-warning and monitoring systems in real lending operations watch delinquency rates closely: rising delinquency today often means rising defaults in a few months. This project's data does not include a delinquency history, which is the specific reason a true early-warning system isn't built here — you can't monitor a signal you don't have.

Recovery rate

Of the money owed on a loan that has defaulted, the percentage the lender actually manages to claw back — through partial payments made before the charge-off, and through collections efforts afterward. Recovery rate and LGD are two sides of the same coin: $LGD = 1 - \text{recovery rate}$. This project computes the recovery rate directly from real data (summing actual principal repaid plus recoveries, divided by the amount originally funded, across every loan that charged off) rather than assuming an industry-standard figure, which is a meaningfully more rigorous approach than most student-level credit projects take.

Grade / sub-grade

Lending Club's own internal risk classification assigned to every loan at origination — a letter from A (safest) to G (riskiest), further subdivided into numbered sub-grades. It functions like a credit rating specific to that platform, and it directly determines the interest rate charged. Crucially, this project excludes grade and interest rate from the model's own inputs, because they're already the OUTPUT of Lending Club's own internal risk assessment — feeding a model its own conclusion as an input would be circular. Grade is still useful, just for a different job: as an independent benchmark to check the new model against ('does our model beat what the lender was already doing'), and as a dimension to segment other results by, like the LGD-by-grade breakdown.

DTI — Debt-to-Income ratio

A borrower's total monthly debt obligations divided by their monthly income, usually expressed as a percentage. It's one of the oldest and most standard affordability signals in all of consumer lending: a borrower already spending 40% of their income on existing debt has much less room to absorb a new payment than one spending 10%. In this project's model, DTI consistently ranks among the strongest predictors of default (second only to credit score in the real feature-importance analysis), which matches what decades of lending practice already assumed.

Revolving utilisation

The share of a borrower's available revolving credit — mainly credit cards — that's currently in use. A borrower using \$9,000 of a \$10,000 credit limit has 90% utilisation; one using \$1,000 of the same limit has 10%. High utilisation is one of the classic early signs of financial distress (people tend to lean harder on available credit right before things go wrong), which is why it's a meaningful input to the model here, distinct from DTI (which looks at required payments) and distinct from the raw revolving balance (which doesn't account for how much credit was actually available).

FICO score

The most widely used consumer credit score in the United States, a three-digit number (roughly 300-850) summarizing a person's credit history into a single figure lenders use as a quick risk signal. Lending Club's data reports a FICO RANGE for each applicant rather than one exact number (for privacy/reporting reasons), so this project uses the midpoint of that range as a single usable value. It's worth knowing that FICO itself is already a model — a commercial one, built by a company called FICO — so this project's own PD model is, in effect, building a second, more specific risk estimate that USES FICO as one of many inputs, rather than replacing it outright.

Basis point (bp)

A standard unit in finance equal to one hundredth of a percentage point (0.01%) — so 100 basis points equals 1%. It exists because percentage-point differences in rates and spreads are often small enough that talking in whole percentages is too imprecise; saying a rate moved '25 basis points' is more exact than saying it moved 'a quarter of a percent,' even though they mean the same thing. This project's servicing-cost and cost-of-funds assumptions (0.4% and 4% per year respectively) would typically be quoted in basis points in a real finance context (40bp and 400bp).

Amortisation

The process by which a loan's principal balance is gradually paid down over its term through a schedule of regular payments, each one covering some interest and some principal. Early in a loan's life, a larger share of each payment goes to interest; later payments pay down more principal. This project's profit simulator uses an 'amortisation factor' (0.52) as a simplifying shortcut: instead of modelling the exact month-by-month schedule, it assumes the average outstanding balance over a 36-month loan's life is roughly 52% of its original amount — a reasonable approximation, but an approximation nonetheless, which is part of why the profit figures are described as directional rather than precise.

Principal

The original amount of money borrowed, before any interest is added — as distinct from the interest, which is the extra cost charged on top for the privilege of borrowing it. When a loan is 'paid down,' what's actually being reduced is the outstanding principal balance; the interest is a separate, ongoing charge calculated against whatever principal still remains.

Cost of funds / funding cost

What it actually costs a lender to get hold of the money it then lends out to borrowers — for a bank, roughly the interest it pays to depositors or bondholders; for a platform like Lending Club, the return it has to offer the investors funding the loans. A lender's profit margin on any loan is fundamentally the difference between the interest rate it charges the borrower and its own cost of funds, minus expected losses and operating costs. This project models cost of funds as a flat 4% annual assumption in the profit simulator.

Servicing cost

The ongoing, per-loan operational cost of actually administering a loan over its life — billing, payment processing, customer service, statements, and so on — as distinct from the cost of the money itself (funding cost) or the cost of it going bad (credit loss). It's a real, if often underestimated, drag on profitability. This project's own methodology notes flag that the specific servicing-cost assumption used (0.4% of principal per year) was chosen partly because a more conservative earlier estimate (1.2%/year) made even the safest loans in the backtest look unprofitable — a good example of how sensitive a profit calculation can be to an assumption that's easy to overlook.

Portfolio

The entire collection of loans a lender currently holds on its books at a given point in time — the thing whose AGGREGATE risk (not any single loan's risk) is what portfolio-level analytics like this project's dashboard actually measure. Questions like 'how much money is at risk across the whole book' or 'where does our expected loss concentrate' are portfolio-level questions, distinct from 'is this one applicant risky,' which is an underwriting-level question.

Vintage

The period — typically the calendar year — in which a group of loans was originated. Grouping loans by vintage and comparing how each group has performed is one of the most standard diagnostic views in consumer lending, because it separates two very different explanations for rising losses: is the BOOK getting riskier (newer vintages performing worse than older ones), or is an OLD cohort simply seasoning normally as it ages? This project's vintage breakdown shows default rates climbing gently from the 2012 to 2015 originations, which is worth investigating further rather than assuming it means the underwriting standard slipped.

Risk band

A small number of coarse buckets (this project uses Low / Medium / High / Critical) that a continuous PD score gets sorted into, purely to make the number easier for a human to read and act on at a glance. A PD of 4.9% and a PD of 21% are both individually precise numbers, but 'Low' versus 'Critical' communicates the difference instantly to someone scanning a dashboard rather than reading a decimal. The cut-offs that define each band are a policy choice, not a law of nature — this project makes that explicit by letting them be adjusted in Settings, clearly separate from the actual underwriting approval threshold used in real decisions.

Adverse-impact ratio (AIR)

The core number in a disparate-impact fairness check: take the approval rate of the group with the LOWEST approval rate on some dimension (say, income band), divide it by the approval rate of the group with the HIGHEST approval rate, and you get a ratio between 0 and 1. A ratio of 1.0 means every group is approved at exactly the same rate; a low ratio means one group is approved far less often than another. This project measured an AIR of 0.42 for income band, meaning the lowest-income group was approved at only 42% of the rate of the highest-income group — a large, and statistically confirmed, gap.

4/5ths rule

A long-standing, widely-used regulatory rule of thumb (originating from US equal-employment-opportunity enforcement, but commonly applied to lending too): if a policy's adverse-impact ratio for some group falls below 0.80 (i.e. that group is approved at less than four-fifths the rate of the most-favoured group), it's treated as a signal worth investigating for discriminatory effect, even if the policy never explicitly considered that group at all. It's a screening threshold, not a legal verdict on its own — but failing it is exactly the kind of finding a real compliance function would want to catch internally, before a regulator or a lawsuit finds it first.

Disparate impact

A legal and analytical concept distinct from intentional discrimination: a policy can be completely neutral on its face, never even referencing a protected characteristic, and still produce a meaningfully worse outcome for one group than another. Disparate impact analysis asks about the OUTCOME of a policy, not the INTENT behind it — which is exactly why it has to be checked for explicitly (as this project's fairness module does) rather than assumed away just because a model was never given a protected attribute to discriminate on directly.

Reason codes / adverse-action notice

When a US lender declines a credit application, the law requires them to tell the applicant the specific reasons why — these are 'reason codes,' and the notice containing them is the 'adverse-action notice.' In a model-driven underwriting system, generating a true reason code means being able to explain, for that ONE specific applicant, which factors pushed their score into decline territory and by how much — a genuinely harder problem than just explaining what the model cares about in general. This project's real per-loan attribution sample is a step in that

direction, but is explicitly not a production-grade version of this requirement.

ECOA / Regulation B

The Equal Credit Opportunity Act is the core US federal law prohibiting credit discrimination based on protected characteristics like race, sex, age, and national origin; Regulation B is the set of detailed rules the Consumer Financial Protection Bureau uses to actually implement and enforce it, including the adverse-action-notice requirement above. This project references ECOA specifically to be honest about a real limitation: a genuine ECOA-style fair-lending test requires data on those protected characteristics, and the public dataset used here simply doesn't have any — so what's built instead is described precisely as a proxy check, not the real thing.

Model risk (management)

The recognition that a risk model is not just a tool but a potential SOURCE of risk in its own right — if it's wrong, stale, poorly understood, or ungoverned, it can cause real financial and legal harm while looking authoritative the whole time. Model risk management is the discipline built around that recognition: independent validation before a model goes live, ongoing monitoring after it does, clear documentation of its assumptions and limitations, and a defined process for retiring or retraining it when it stops working. This project deliberately borrows several of these practices (a real out-of-time test, a tested benchmark, documented limitations) without claiming to be a full model-risk-management program, which would also need independent review this project doesn't have.

Statistics & machine-learning terms

AUC / ROC curve

AUC (area under the ROC curve) measures how well a model ranks risky cases above safe ones, on a scale from 0.5 (no better than flipping a coin) to 1.0 (perfect separation, every risky case scored above every safe one). Concretely: if you picked one defaulter and one non-defaulter at random from the test set, AUC is the probability the model assigns the defaulter a higher risk score. The ROC curve itself is a plot of true-positive rate against false-positive rate as you slide the decision threshold from one extreme to the other; AUC is just the area underneath that curve, condensed into one number. It's the most commonly quoted headline metric in credit scoring, but it's a RANKING metric — it says nothing about whether the model's actual probability numbers are trustworthy, which is what calibration checks separately.

Gini coefficient

A simple rescaling of AUC onto a different, more intuitive 0-to-1 discrimination scale that's traditionally more common in European and Indian credit-risk practice than AUC is: $\text{Gini} = 2 \times \text{AUC} - 1$. An AUC of 0.69 (this project's real test-set result) corresponds to a Gini of 0.38. The two numbers always move together and measure exactly the same underlying thing — a model's ability to separate risky from safe cases — they're just reported on different scales depending on which convention a given team or region uses.

KS statistic (Kolmogorov-Smirnov)

A second, classic way of measuring how well a model separates good and bad outcomes, distinct from AUC. Sort the population by predicted risk score, then track two running (cumulative) percentages as you move down that sorted list: what share of all the GOOD loans you've passed, and what share of all the BAD loans you've passed. KS is simply the largest gap that ever opens up between those two lines. Where AUC summarises separation across the WHOLE range of the score at once, KS highlights the single point where the model's discrimination is strongest — both are reported together in this project's Model Risk section because they can tell slightly different stories about the same model.

Calibration

Whether a model's stated probabilities are actually true on average, as opposed to just correctly RANKING risk. A model can have excellent AUC (it correctly puts the riskiest loans at the top of the list) while being badly calibrated (it says '60% chance of default' for loans that actually default only 20% of the time). Calibration matters enormously here because Expected Loss is a DOLLAR figure computed directly from the model's raw PD number — if the probabilities themselves aren't trustworthy, neither is any dollar figure built on top of them. This project checks calibration with a decile table (below) and explicitly notes where it breaks down: the highest-risk decile is measurably under-predicted, a real, documented limitation rather than a hidden one.

Decile / lift

A calibration-checking technique: sort the scored population by predicted risk from safest to riskiest and split it into ten equal-sized groups (deciles). For each decile, compare the model's own average predicted PD against the ACTUAL observed default rate in that group — a well-calibrated model should see those two numbers track closely in every decile, not just on average across the whole population. 'Lift' describes how much better (or worse) a given decile performs relative to the overall base rate — the riskiest decile in this project shows roughly 2.2x the portfolio's average default rate, which is exactly the kind of separation a useful risk model should produce.

DeLong test

A specific statistical test (named after its authors) for comparing the AUCs of two models scored on the exact same sample, answering a precise question: is Model A's AUC really higher than Model B's, or could a difference this size have happened just by chance on this particular sample of loans? It returns a z-statistic and a p-value. This project uses it to test the trained model against a much simpler baseline (Lending Club's own risk grade) — the test came back with p roughly equal to 3×10^{-85} , an extraordinarily small number that means the model's edge over the simple baseline is real, not a fluke of this specific dataset.

p-value / statistical significance

A number, between 0 and 1, that answers: if there were truly NO real effect or difference at all, how surprising would the result we actually observed be, purely from random chance? A small p-value (conventionally below 0.05) means the observed result would be quite surprising under a 'nothing is really going on' assumption, which is treated as evidence that something real IS going on. It's easy to misread a p-value as 'the probability the finding is true' — it isn't that; it's a statement about how compatible the data is with the assumption of no effect, nothing more, nothing less.

Bootstrap / bootstrapping

A way of estimating how much uncertainty surrounds a statistic, without needing a complicated mathematical formula for it. The trick: repeatedly draw new samples from your existing data (sampling WITH replacement, so the same row can be picked more than once in one draw), recompute the statistic of interest on each resampled version, and look at how much the answer varies across, say, 2,000 repeats. That spread of results IS your estimate of uncertainty. This project bootstraps the fairness adverse-impact ratio exactly this way — resampling each demographic group's approval outcomes 2,000 times to see how stable that ratio really is, rather than trusting a single point estimate.

Confidence interval (CI)

A range of plausible values for some true, unknown number, built from the data's own uncertainty — the direct output of a bootstrap (or of a formula-based method). A '95% confidence interval' is loosely read as 'a range we're reasonably confident contains the true value.' This project reports a confidence interval on its adverse-impact ratio findings specifically so a reader can see not just the single number (0.42 for income band) but how much that number could plausibly move around given normal sampling variation — the interval's upper bound staying safely below the 0.80 fairness threshold is what makes the finding a real one, not noise.

Bonferroni correction

A standard, deliberately conservative adjustment applied when you're testing more than one hypothesis at the same time, to avoid a subtle trap: if you run enough independent tests, SOME of them will look statistically significant purely by chance, even if nothing real is happening in any of them. The Bonferroni correction tightens the significance bar in proportion to how many tests you're running — this project tests fairness across three dimensions at once (income, region, home ownership), so it divides its significance threshold by three before deciding whether a finding counts as real, which is a stricter, more honest bar than testing each dimension in isolation would be.

Credibility weighting (Buhlmann)

A technique from actuarial science for handling a very specific, common problem: what do you do with a segment of your data that's too small to trust fully on its own, but not so small that you should ignore it entirely? Buhlmann credibility weighting blends that small segment's own raw estimate with a larger, more reliable population-wide estimate, in a proportion determined by how much data actually supports the segment — a segment with abundant data gets weighted almost entirely toward its own number, while a very thin segment gets pulled mostly toward the population average. This project applies it to LGD by grade: Grade G has only 73 charged-off loans to learn from, so its credibility-weighted LGD (about 50%) sits much closer to the flat portfolio figure than its own raw, noisy estimate (60%) would suggest on its own.

Class imbalance

A dataset where one outcome is much more common than the other — here, roughly 85% of loans are 'good' (fully paid) and about 15% are 'bad' (defaulted). This matters because a lazy model can score deceptively well on a naive metric like plain accuracy just by predicting the majority class every time, without having learned anything useful. It's why this project deliberately avoids accuracy as a headline metric in favour of AUC, KS, and calibration, all of which remain meaningful even with a skewed base rate — and why the decision to NOT artificially rebalance the training data is explained explicitly in the code rather than left as an unexplained default.

Overfitting

When a model learns the specific quirks and noise of the exact data it was trained on, rather than a genuine, generalisable pattern — the model equivalent of memorising the answers to one specific practice exam instead of actually learning the subject. An overfit model looks impressive on the data it already knows and performs noticeably worse on anything new. The single best defence against it is evaluating a model on data it never saw during training, which is exactly what an out-of-time test split (below) is for.

Out-of-time (OOT) split

Splitting data into a training set and a test set based on TIME rather than a random shuffle — here, loans issued before 2015 train the model, loans issued from 2015 onward test it. This is the more honest, more realistic validation approach for anything that will eventually be used to score the future, because that's exactly the condition the model will face in real deployment: it will only ever be asked to score loans it hasn't seen yet, from later than its training data. A random split would let information from the same time period leak between train and test in subtle ways, making the model look more reliable than it really would be on genuinely new, future applicants.

Cross-validation

A technique for getting a more reliable performance estimate than a single train/test split provides, by splitting the data into several equal folds, then repeatedly training on all-but-one fold and testing on the held-out one, rotating which fold is held out each time. This project uses a light version of the idea internally, inside the calibration step (CalibratedClassifierCV with 3 folds), specifically so the model's probability calibration is learned on data the base model itself didn't get to train on directly — avoiding the calibrator simply memorising the same data the underlying model already fit to.

Feature engineering

The practice of deriving new input variables from raw data specifically to make a pattern easier for a model to detect, rather than leaving the model to infer that pattern from more basic inputs on its own. This project's clearest example: computing 'loan-to-income ratio' by dividing loan amount by annual income, rather than feeding the model those two raw numbers separately and hoping it discovers the ratio matters — an engineered feature like this often captures a real-world relationship far more directly and efficiently than raw inputs alone would.

One-hot encoding

A standard way of turning a categorical variable (something with named categories rather than numbers — like loan purpose, which might be 'credit_card,' 'debt_consolidation,' 'medical,' and so on) into a form a numeric model can actually use: create one new 0/1 column per category, and mark a 1 in whichever column matches that row's actual value, 0 everywhere else. It's called 'one-hot' because exactly one of those new columns is 'hot' (1) for any given row. This project applies it inside the model's preprocessing pipeline automatically, and also caps how many rare categories get their own column (via a minimum-frequency setting) so an extremely uncommon category doesn't blow up the number of columns for little benefit.

Gradient boosting

A machine-learning technique that builds a strong predictive model out of many small, individually weak decision trees, added one at a time in sequence, where each new tree is specifically trained to correct the mistakes the trees before it were still making. It's one of the most consistently strong-performing methods on the kind of structured, tabular, spreadsheet-shaped data credit risk problems typically involve (as opposed to images or text, where other methods tend to dominate). This project's primary model uses scikit-learn's HistGradientBoostingClassifier, a fast, histogram-based implementation of the same core idea.

Isotonic regression / calibration

A flexible, non-linear statistical technique used here specifically to CALIBRATE a model's raw output scores into genuine, trustworthy probabilities. Unlike a simpler calibration method that assumes a fixed mathematical shape (like a straight line or an S-curve) relating raw score to true probability, isotonic regression only assumes the relationship is monotonic (higher raw score always means higher true risk, in whatever shape that relationship actually takes) and lets the data determine everything else — a good fit when there's enough data to support it, which this project's

several-hundred-thousand-loan training set comfortably has.

Permutation importance

A direct, empirical way of measuring how much a trained model actually relies on one particular input feature, without needing to understand the model's internal mechanics: take that one feature's column, shuffle its values randomly across all the rows (destroying any real relationship it had to the outcome while keeping everything else about the data intact), and measure how much the model's performance (AUC, here) drops as a result. A feature the model barely uses will barely move the score when shuffled; a feature it depends on heavily will cause a real drop. This project runs this directly on the actual trained model, which is a real, model-specific finding rather than a generic assumption about what 'should' matter.

SHAP (Shapley values)

A rigorous method, borrowed from cooperative game theory, for explaining one SPECIFIC prediction by fairly splitting the credit for that prediction among all of the input features that contributed to it. The 'fair' part is mathematically precise: it comes from averaging each feature's marginal contribution across every possible ORDER the features could have been considered in, which is a genuinely combinatorial calculation. This project's own per-loan explanation is explicitly and deliberately NOT full SHAP — it perturbs one feature at a time from a single fixed baseline rather than averaging over every ordering — a simpler, real, useful calculation, but a different and less complete one, and it's labelled that way on purpose rather than borrowing SHAP's name for something that isn't quite SHAP.

Data leakage

The single most common and most dangerous mistake in building a predictive model: information that wouldn't actually have been available at the moment a real prediction needs to be made somehow ends up in the training data anyway, making the model look far more accurate than it will ever be in real use. The classic credit-risk example, and the one this project is built specifically to avoid: including a loan's final interest rate or the lender's own risk grade as a model input, when those were themselves already SET based on an assessment of the loan's risk — the model would just be learning to reverse-engineer a decision that had already been made, not predicting risk independently.

Engineering & software terms

Pipeline

An ordered sequence of processing stages where each stage's output becomes the next stage's input, run in a fixed, repeatable order. This project's pipeline runs raw data through cleaning, feature/outcome separation, model training, scoring, expected-loss calculation, and finally analysis (backtest, fairness, monitoring) — fourteen named stages in total, each one small and individually testable rather than one giant, unreadable script that does everything at once. The word 'pipeline' is used deliberately to evoke that plumbing metaphor: data flows through in one direction, and each stage should be understandable on its own without having to read every other stage first.

ETL

Extract, Transform, Load — a decades-old, still-standard pattern for moving data from a source, reshaping it into a usable form, and depositing it somewhere it can actually be queried and analysed. This project's first three pipeline stages are a textbook example: EXTRACT the raw CSV into DuckDB, TRANSFORM it through typing/cleaning/feature-splitting, and effectively LOAD it into the clean, query-ready tables everything downstream reads from. Understanding a project as 'an ETL pipeline plus a model on top' is a genuinely useful way to describe what's happening here in one sentence.

DuckDB

An embedded, file-based SQL database engine — meaning it runs directly inside your own program with no separate server process to install or manage, similar in spirit to SQLite, but specifically built and optimised for fast ANALYTICAL queries (aggregations, joins, scans across millions of rows) rather than the many-small-transactions workload a typical app database handles. It's a good fit for a project like this one: all the data fits comfortably in a single local file, there's no need for multiple simultaneous users, and the queries involved (group-bys, joins across a few hundred thousand rows) are exactly the kind of workload DuckDB is fast at.

Mart (data mart)

A focused table or view built to serve one specific analytical purpose, as opposed to a giant, do-everything table trying to answer every possible question at once. This project's `mart_loan_features`, `mart_loan_benchmark`, and `mart_loan_outcomes` are a clean example: each one exists for a distinct reason (what the model may see, what the benchmark uses, what the real outcome was), and keeping them as separate marts rather than one wide table is precisely what makes the leakage-prevention architecture actually work — a column that isn't in a mart physically cannot be read from it.

Bronze / silver / gold

A common naming convention in modern data-engineering for describing pipeline stages by how refined the data is at each point: BRONZE is raw, as-received data, untouched and unvalidated; SILVER is cleaned, typed, deduplicated data with obvious quality problems handled; GOLD is business-ready, often aggregated data, shaped for a specific analytical or reporting purpose. This project's own stage comments describe themselves in almost exactly these terms — stage 1 (raw ingestion) is bronze, stage 2 (staging/cleaning) is silver, and stage 3 onward (the feature/outcome marts, the serving marts) is gold.

Star schema

A standard way of organising a BI (business intelligence) data model, named for the shape it makes in a diagram: one central FACT table holding the numeric measures you want to analyse (in this project, `FACT_LOAN` — loan amount, predicted PD, expected loss, one row per loan), surrounded by smaller DIMENSION tables holding the descriptive context you slice those numbers by (`DIM_DATE`, `DIM_BORROWER`, `DIM_LOAN_PRODUCT`). It's the standard shape BI tools like Power BI expect and are fastest at querying, which is exactly why this project builds one as a dedicated view layer, separate from the more normalised tables the Python pipeline itself uses internally.

Artifact

In this context, a saved output file produced by one run of the pipeline that something downstream — another pipeline stage, or the dashboard — reads from afterward, rather than recomputing from scratch every time. This project's artifacts are mostly small JSON files (metrics.json, backtest.json, fairness.json, and so on) plus the trained model file itself (model.joblib) — each one a frozen snapshot of a real result, which is exactly what lets the dashboard show real numbers without needing to run Python or query a database at all.

Orchestration

The job of deciding WHICH steps of a multi-stage process run, in WHAT order, and coordinating handing control from one to the next — distinct from the actual logic inside any individual step. This project's run_pipeline.py is purely an orchestrator: it doesn't itself clean data, train a model, or compute a metric; it just calls, in the right sequence, the real modules (src/train.py, src/lgd.py, and so on) that do that actual work, which keeps the sequencing logic and the business logic cleanly separated and each independently readable.

Reproducibility

The property that re-running a process produces the same (or acceptably close to the same) result every time, rather than something different on each run. It matters enormously for any analytical pipeline, because a result nobody can reproduce isn't really trustworthy, no matter how good it looks the first time. This project checks reproducibility directly and automatically: its final pipeline stage retrains the model from scratch and compares the newly-measured AUC against the previous run's AUC, failing loudly if the two differ by more than a small tolerance (0.005) — turning 'is this still reproducible' from a hopeful assumption into an actual, automated test.

CI / CI pipeline (Continuous Integration)

The practice of automatically running a project's tests (and other checks, like a build or a type-check) every single time code changes, rather than relying on a person to remember to run them manually before something ships. The point is to catch a broken change within minutes of it being introduced, while it's still fresh and easy to fix, instead of discovering it days or weeks later. This project's GitHub Actions workflow runs its full real test suite, an import-sanity-check across every pipeline module, and a full dashboard build, on every single push — something the project didn't have at all before it was added as part of an audit-driven fix.

Seeded / deterministic (random number generation)

Computers can't generate TRULY random numbers on demand the way, say, a dice roll is random — instead they use an algorithm that produces a long sequence of numbers that LOOKS random, starting from some initial starting value called a 'seed.' Give the same seed twice, and you get the exact same 'random' sequence back both times — that's what 'deterministic' means here: repeatable despite looking random. This project relies on that property heavily for its synthetic (DEMO) data: the same seed value (20240611, for example) is what guarantees the 220 synthetic borrowers are identical every time the dashboard is rebuilt, rather than silently changing on every deploy.

Config-driven

A design principle where a program's behaviour is controlled by values sitting in a separate configuration file, rather than hardcoded directly inside the program's logic — so a genuine change in assumption (a different cost-of-funds rate, a different maturity window, a different model hyperparameter) can be made by editing one config value instead of hunting through code. This project's config.yaml is explicit about this aim in its own opening comment ('nothing hard-coded in the pipeline'), though a real limitation worth knowing is that claim only fully holds for THRESHOLDS and assumptions — the actual data SCHEMA (which columns exist, what they're called) is still hardcoded elsewhere, a genuine, stated gap rather than a hidden one.

API call / fetch

A request one piece of software makes to another (often over the internet) asking for data or asking it to do something, then waiting for a response — the basic building block of how most modern web apps talk to a backend server. CREDENCE deliberately makes ZERO of these at runtime: every real number it shows was baked directly into the compiled JavaScript at build time, from files transcribed off the pipeline's own output. That's precisely what makes it a genuinely static site rather than a normal web app with a live backend behind it.

SPA (Single-Page Application)

A style of web application that loads its HTML/JavaScript/CSS once, and from then on updates what's on screen dynamically inside the browser as the user navigates, rather than requesting an entirely fresh page from a server for every click the way older, traditional websites do. It tends to feel snappier once loaded, since only the data that actually changed needs to update, not the whole page. CREDENCE is a textbook SPA, built with React — every page transition you see is the same already-loaded JavaScript swapping out what it renders, not a new page load.

Hash routing

A specific technique for handling page navigation inside a single-page application, using the part of the URL after the # symbol (e.g. `yoursite.com/#!/portfolio`) to represent which 'page' is currently showing, instead of a normal-looking URL path. The real advantage: because a browser never actually sends the part after # to the server on a page load, a hash-routed app works correctly on literally any static file host with zero server-side configuration — no special rewrite rules needed to make a direct link to `/#!/portfolio` load correctly, which is exactly why this project chose it, given the goal of deploying to a plain static host like GitHub Pages.

JSON

JavaScript Object Notation — a simple, human-readable text format for representing structured data (nested objects, lists, numbers, strings, true/false values) that's become the near-universal standard for exchanging data between different pieces of software, regardless of what programming language either side is written in. Every one of this project's pipeline artifacts (`metrics.json`, `fairness.json`, and so on) is written in this format specifically because it's simple, widely supported, and easy for both a human and another program to read.

Static site / static hosting

A website that's served to visitors as a set of pre-built, unchanging files (HTML, CSS, JavaScript) with no backend server generating anything dynamically on the fly for each request — as opposed to a dynamic site, which runs server-side code (often talking to a database) to build each page freshly. A static site is generally cheaper, faster, and simpler to host and secure, precisely because there's no server-side logic or database to run, patch, or worry about being attacked. CREDENCE is fully static, which is exactly why it can be deployed to a free host like GitHub Pages with nothing more than a folder of built files.